

Design and Implementation of a Database Management System for Pharmacy Prescription and Medicine Stock Management System

Course Code: CSE2101 — Database Management Systems

Program: Bachelor of Technology (B.Tech) in Computer Science and Engineering

Academic Year: 2026–2027 | **Semester:** Semester III

Department: Department of Computer Science & Engineering, School of Technology

Institution: Woxsen University, Hyderabad, Telangana, India

Candidate Name	Roll Number
Y. Abhinav Sri Ram	25WU0101158

Under the Guidance and Supervision of:

Dr. Kiran Mayee Advala

Associate Professor, Department of Computer Science & Engineering

School of Technology, Woxsen University, Hyderabad, Telangana, India

2. Abstract

Modern retail and clinical pharmacy workflows encounter persistent operational challenges regarding manual inventory audits, accidental dispensing of expired pharmaceutical lots, and non-compliance with statutory drug distribution mandates. This project presents **AuraCare Rx**, an enterprise-grade Relational Database Management System (RDBMS) paired with an interactive clinical web application designed to govern pharmacy operations.

The database architecture is implemented in MySQL 8.0 (InnoDB Storage Engine), comprising twelve normalized relational tables structured in Third Normal Form (3NF) to systematically eradicate data redundancy and insertion, update, and deletion anomalies. The system incorporates declarative referential integrity via strict foreign key constraints and an active transactional trigger, `EnforcePrescriptionRule`, which intercepts sales transactions at the database engine level to disallow the dispensing of restricted Schedule H drugs without an authorized physician prescription. A modern, three-role Role-Based Access Control (RBAC) architecture segregates operational duties across Pharmacists, Inventory Managers, and Administrators. The platform was evaluated using SQL stress tests, multi-table analytical joins (e.g., supplier procurement traces and automated 30-day near-expiry warning ledgers), and full CRUD verification, demonstrating zero orphaned data states, deterministic query execution, and strict regulatory compliance.

3. Introduction & Problem Statement

3.1 Background and Industry Context

The dispensing and distribution of pharmaceuticals represents a highly sensitive intersection of logistics, public health, and statutory compliance. Pharmacies handle thousands of Stock Keeping Units (SKUs)

spanning broad therapeutic and chemical classifications. Each medication is bound to stringent quality controls, distinct batch identifiers, manufacturing lots, wholesale distributor pipelines, and finite shelf-life spans.

Historically, community pharmacies and dispensaries have relied heavily on manual physical paper logbooks or non-relational spreadsheet applications (e.g., Microsoft Excel). While accessible, spreadsheets fail to provide transactional atomicity, concurrency control, domain constraints, or referential integrity. As operational scale increases, manual tracking introduces critical systemic vulnerabilities.

3.2 Problem Statement

Manual and decentralized pharmacy workflows present three critical vulnerabilities:

1. **Batch Blindness and Financial Leakage:** When medicine stock is tracked only as aggregate unit counts rather than at the discrete lot/batch level, older stock is obscured by incoming stock. Expired or degrading medications remain on shelves alongside fresh stock, risking patient harm, product recalls, and severe inventory write-offs.
2. **Regulatory Non-Compliance in Dispensing:** Controlled pharmaceuticals (such as Schedule H, Schedule X, and narcotic substances) are legally prohibited from being dispensed without a validated doctor's prescription. In manual or spreadsheet-based billing systems, verification depends entirely on human vigilance at checkout, frequently failing regulatory audits and exposing pharmacies to legal prosecution.
3. **Referential Inconsistencies and Mutation Anomalies:** Non-normalized schemas duplicate customer, doctor, supplier, and drug details across multiple sheets. This results in update anomalies (e.g., changing a supplier contact leaves historical orders inconsistent), insertion anomalies (e.g., inability to catalog a drug category before acquiring stock), and deletion anomalies (e.g., deleting a customer billing row erases the doctor's prescription record or inventory master).

4. Objectives & Scope

4.1 Core Objectives

1. **Architect a 3NF Relational Model:** Design and implement a 12-table relational schema in MySQL 8.0 that eliminates data anomalies and achieves Third Normal Form (3NF) compliance.
2. **Enforce Database-Level Clinical Safeguards:** Implement stored triggers (EnforcePrescriptionRule) to block unauthorized dispensing of restricted drugs at the database engine level, ensuring policy enforcement independent of client-side validation.
3. **Implement Real-Time Batch Lifecycle Tracking:** Model inventory through discrete lots (Batches), tracking stock intake, expiration dates, unit costs, and movement histories.
4. **Deploy Role-Based Web Workspaces (RBAC):** Deliver an administrative and operational web interface with 3-role permission segregation (Pharmacist, Inventory Manager, Administrator) providing View, Insert, Update, and Delete operations.
5. **Optimize Analytical Decision-Making:** Facilitate analytical SQL joins to report supplier performance (e.g., "Metro Meds" deliveries) and automated near-expiry alert summaries within 30-day thresholds.

4.2 Project Scope

Functional Domain	In-Scope Components	Out-of-Scope System Boundaries
Catalog & Procurement	Master drug data (Medicines, Categories, Manufacturers), Wholesale Suppliers, Purchase Consignments (Purchases).	Physical automated robotic warehouse dispensing systems and automated pallet logistics.
Inventory Governance	Granular physical batches (Batches), dynamic shelf counts, expiry tracking, and audit ledgers	Temperature/humidity IoT sensor telemetry monitoring cold-chain storage refrigeration.

Functional Domain	In-Scope Components	Out-of-Scope System Boundaries
	(Stock_Movements).	
Clinical & Dispensing	Customer profiles, Clinical Prescriptions, Outward Invoices (Sales), Line Items (Sale_Items), Settlement Logging (Payments).	External Electronic Health Record (EHR) HL7/FHIR bidirectional clinical exchange protocols.
Access & Security	3-Role persona segregation (Pharmacist, Inventory Manager, Administrator) via application interface.	Multi-tenant payment gateway escrow settlements and direct interbank clearing networks.

5. Software & Hardware Requirements

5.1 Software Requirements

Layer / Component	Technology Selected	Version / Specification	Functional Purpose
Operating System	Ubuntu Linux / Windows / macOS	Ubuntu 22.04 LTS / Win 10/11 / macOS 15+	Host environment for server and database runtimes
DBMS Engine	MySQL Community Server	Version 8.0.30+ (InnoDB Engine)	Relational storage, foreign keys, triggers, ACID engine
Database Tooling	MySQL Workbench & CLI	Workbench 8.0 CE / Native CLI	Schema management, query profiling, administration
Backend Runtime	Python / Flask Framework	Python 3.10+ / Flask 2.3+	RESTful microservice routing, business logic, JSON API
Database Driver	mysql-connector-python	Version 8.0+ (Oracle Driver)	Thread-safe connection pooling and parameterized execution
Presentation Layer	HTML5 / Vanilla JavaScript (ES6+)	W3C Standard / Fetch API	Dynamic, asynchronous client-side DOM manipulation
Styling & Icons	Tailwind CSS & Lucide	Tailwind v3.3+ (CDN) /	Responsive clinical UI

Layer / Component	Technology Selected	Version / Specification	Functional Purpose
	Icons	Lucide v0.280+	layout, modals, and status badges
Version Control	Git & GitHub	Git v2.40+ / Cloud VCS	Source code versioning, collaboration, tracking

5.2 Hardware Requirements

Hardware Resource	Minimum System Specification	Recommended Production Specification
Processor (CPU)	Intel Core i3 / AMD Ryzen 3, Dual-Core 2.0 GHz	Intel Core i5 / AMD Ryzen 5 (Quad-Core ≥ 2.4 GHz)
System Memory (RAM)	4 GB DDR4	16 GB DDR4/DDR5 (for concurrent buffer pools and servers)
Storage (Disk)	500 MB free space	256 GB PCIe NVMe M.2 Solid State Drive (SSD)

Hardware Resource	Minimum System Specification	Recommended Production Specification
Network	10/100 Mbps Fast Ethernet	Gigabit Ethernet (1000BASE-T) or Wi-Fi 6 (802.11ax)
Display Resolution	1366 × 768 WXGA Standard	1920 × 1080 Full HD (1080p) Clinical Workspace Display

6. Entity-Relationship (ER) Model

6.1 Entity Identification and Descriptions

1. **Categories:** Therapeutic and chemical classifications of drugs (e.g., Antibiotics, Analgesics, Antipyretics).
2. **Manufacturers:** Licensed pharmaceutical companies producing drug formulations.
3. **Suppliers:** Wholesale vendors, distribution agencies, and clearinghouses supplying stock.
4. **Customers:** Patients and retail consumers purchasing medications.
5. **Prescriptions:** Official clinical documents authorized by medical practitioners.
6. **Purchases:** Inward wholesale procurement orders received from suppliers.
7. **Medicines:** Master drug catalog records defining chemical formulations, therapeutic classes, and Schedule H flags.
8. **Batches:** Granular physical inventory lots with production codes, expiry dates, quantities, and acquisition costs.
9. **Sales:** Outward transactional retail sales invoices and billing encounters.

10. **Sale_Items:** Child line items specifying the exact batch, unit count, and dispensed price for each item in a sale.
11. **Payments:** Financial reconciliation records capturing settlement methods and reconciled currency amounts.
12. **Stock_Movements:** Audit ledger tracking physical inventory adjustments, inward stock, and dispensing deductions.

6.2 Structural Cardinalities & Relationships

- **Categories to Medicines (1:N):** One therapeutic category encompasses many catalog medicines; each medicine belongs to exactly one category.
- **Manufacturers to Medicines (1:N):** One manufacturer produces many catalog medicines; each medicine is manufactured by exactly one pharmaceutical producer.
- **Suppliers to Purchases (1:N):** One wholesale supplier fulfills multiple purchase orders over time; each purchase consignment references one supplier.
- **Purchases to Batches (1:N):** One inward purchase consignment introduces one or more distinct medicine batches; each batch originates from one purchase order.
- **Medicines to Batches (1:N):** One master medicine catalog entry has multiple physical inventory lots produced over time; each batch links to one medicine.
- **Customers to Prescriptions (1:N):** A registered customer/patient presents zero, one, or multiple doctor prescriptions; each prescription belongs to one customer.
- **Customers to Sales (1:N):** One customer executes multiple transactional sales visits; each sales invoice is billed to one customer profile.
- **Prescriptions to Sales (1:N):** A prescription may be utilized across multiple authorized refills; an over-the-counter sale retains a NULL prescription reference.
- **Sales to Sale_Items (1:N):** One sales invoice aggregates one or many line items; each line item belongs to one sales invoice.
- **Batches to Sale_Items (1:N):** One inventory batch provides units across multiple retail sale items; each line item references one physical batch.
- **Sales to Payments (1:1):** One completed sales invoice corresponds to exactly one reconciled fiscal payment record; each payment satisfies one sales invoice.
- **Batches to Stock_Movements (1:N):** One inventory batch has multiple recorded audit

movements across its lifecycle; each movement entry logs the delta of one batch.

 **[FIGURE 6.1 — Complete Chen's Notation ER Diagram]**

Conceptual schema diagram illustrating all 12 normalized entity sets (rectangular boxes), entity attributes (elliptical nodes with underscored primary keys), relationship sets (diamonds), and explicit structural cardinality ratios (1:1, 1:N) enforcing referential constraints across the database system.

7. Relational Schema & Normalization

7.1 Relational Schema Definitions

The conceptual ER model maps directly into twelve relational schemas. Primary keys are marked with **[PK]** and foreign keys are designated as **[FK → TargetTable(TargetColumn)]**:

1. Categories (CategoryID **[PK]**, CategoryName)
2. Manufacturers (ManufacturerID **[PK]**, ManufacturerName)
3. Suppliers (SupplierID **[PK]**, SupplierName)
4. Customers (CustomerID **[PK]**, CustomerName, Phone)
5. Prescriptions (PrescriptionID **[PK]**, CustomerID **[FK → Customers(CustomerID)]**, DoctorName, IssueDate)
6. Purchases (PurchaseID **[PK]**, SupplierID **[FK → Suppliers(SupplierID)]**, PurchaseDate, TotalAmount, OrderStatus)
7. Medicines (MedicineID **[PK]**, CategoryID **[FK → Categories(CategoryID)]**, ManufacturerID **[FK → Manufacturers(ManufacturerID)]**, MedicineName, IsRestricted)
8. Batches (BatchID **[PK]**, MedicineID **[FK → Medicines(MedicineID)]**, PurchaseID **[FK → Purchases(PurchaseID)]**, BatchNumber, ExpiryDate, StockQuantity, UnitCost)
9. Sales (SaleID **[PK]**, CustomerID **[FK → Customers(CustomerID)]**, PrescriptionID

[FK $\rightarrow$ Prescriptions(PrescriptionID)], SaleDate, GrandTotal)

10. Sale_Items (SaleItemID [PK], SaleID [FK $\rightarrow$ Sales(SaleID)], BatchID [FK $\rightarrow$ Batches(BatchID)], Quantity, UnitPrice, Discount, SubTotal)

11. Payments (PaymentID [PK], SaleID [FK $\rightarrow$ Sales(SaleID)], PaymentMethod, AmountPaid)

12. Stock_Movements (MovementID [PK], BatchID [FK $\rightarrow$ Batches(BatchID)], MovementType, Quantity, MovementDate)

7.2 Formal Normalization Analysis (1NF to 3NF)

First Normal Form (1NF) Compliance

Theoretical Definition: A relation R is in 1NF if and only if the domain of each attribute contains only atomic (indivisible) values, and there are no repeating groups, multi-valued attributes, or composite internal arrays.

Proof: In the AuraCare Rx schema, every column is defined using scalar, atomic datatypes (INT, VARCHAR, DECIMAL, DATE, BOOLEAN). No attribute stores concatenated values, multi-element lists, or nested documents. For instance, customer contact numbers are stored as single unique entries, and multi-item orders are not stored as arrays within Sales. Instead, repeating items are normalized into the separate Sale_Items relation. Hence, all relations satisfy 1NF.

Second Normal Form (2NF) Compliance

Theoretical Definition: A relation R is in 2NF if and only if it satisfies 1NF and no non-prime attribute is partially dependent on any candidate key of R . That is, every non-prime attribute must be fully functionally dependent on the entirety of every candidate key.

Proof: A partial dependency exists if and only if there is a candidate key K and a proper subset $X \subset K$ such that $X \rightarrow A$, where A is a non-prime attribute. In AuraCare Rx, all twelve tables utilize **single-attribute surrogate primary keys** (CategoryID, MedicineID, BatchID, SaleID, SaleItemID, etc.). Because every candidate key consists of exactly one attribute ($|K| = 1$), no non-empty proper subset of any candidate key exists. Because proper subsets of candidate keys cannot exist, partial functional dependencies are

mathematically impossible across all relations. Therefore, the entire schema is in 2NF.

Third Normal Form (3NF) Compliance

Theoretical Definition: A relation R is in 3NF if and only if it satisfies 2NF and no non-prime attribute is transitively dependent on the primary key. Formally, for every non-trivial functional dependency $X \rightarrow Y$, at least one of the following conditions must hold:

1. X is a superkey of R .
2. Y is a prime attribute of R (i.e., Y is a component of a candidate key).

Proof via Transitive Dependency Resolution:

- **Resolution of Medicine Catalog:** In an unnormalized design, MedicineName, CategoryName, and ManufacturerName would reside together in a single table, creating transitive dependencies:

MedicineID	→	CategoryID	→	CategoryName
MedicineID	→	ManufacturerID	→	ManufacturerName

In AuraCare Rx, CategoryName and ManufacturerName are factored into distinct entities Categories and Manufacturers. Medicines contains only foreign keys (CategoryID, ManufacturerID), where each determinant is a superkey in its home relation.

- **Resolution of Batch Procurement:** In an unnormalized batch table, supplier details would create:

BatchID	→	PurchaseID	→	SupplierID	→	SupplierName
---------	---	------------	---	------------	---	--------------

This is resolved by establishing the Purchases relation, where SupplierID references Suppliers, ensuring every determinant is a superkey.

- **Resolution of Line Item Calculations:** In Sale_Items, line-item valuations (SubTotal) depend strictly on the line surrogate key SaleItemID, while aggregate totals (GrandTotal) reside in Sales.

Since every functional dependency across all twelve tables satisfies the condition that its determinant is a superkey, the schema is strictly in Third Normal Form (3NF).

8. Data Dictionary

Table 8.1: Categories

Field Name	Data Type	Nullable	Key / Constraint Type	Field Description
CategoryID	INT	NO	PRIMARY KEY, AUTO_INCREMENT	Unique surrogate identifier for the drug category
CategoryName	VARCHAR(100)	NO	UNIQUE, NOT NULL	Canonical therapeutic classification name

Table 8.2: Manufacturers

Field Name	Data Type	Nullable	Key / Constraint Type	Field Description
ManufacturerID	INT	NO	PRIMARY KEY, AUTO_INCREMENT	Unique surrogate identifier for the drug producer
ManufacturerName	VARCHAR(150)	NO	UNIQUE, NOT NULL	Legal commercial corporate name of manufacturer

Table 8.3: Suppliers

Field Name	Data Type	Nullable	Key / Constraint Type	Field Description
SupplierID	INT	NO	PRIMARY KEY, AUTO_INCREMENT	Unique surrogate identifier for wholesale distributor
SupplierName	VARCHAR(150)	NO	UNIQUE, NOT NULL	Commercial trade name of wholesale distributor

Table 8.4: Customers

Field Name	Data Type	Nullable	Key / Constraint Type	Field Description
CustomerID	INT	NO	PRIMARY KEY, AUTO_INCREMENT	Unique surrogate identifier for patient/customer
CustomerName	VARCHAR(150)	NO	NOT NULL	Full legal customer/patient name
Phone	VARCHAR(20)	NO	UNIQUE, NOT NULL	Primary contact telephone number

Table 8.5: Prescriptions

Field Name	Data Type	Nullable	Key / Constraint Type	Field Description
PrescriptionID	INT	NO	PRIMARY KEY, AUTO_INCREMENT	Unique surrogate identifier for prescription record
CustomerID	INT	NO	FOREIGN KEY → Customers(CustomerID)	Target patient reference key
DoctorName	VARCHAR(150)	NO	NOT NULL	Full legal name of the prescribing physician
IssueDate	DATE	NO	NOT NULL	Official date the prescription was authorized

Table 8.6: Purchases

Field Name	Data Type	Nullable	Key / Constraint Type	Field Description
PurchaseID	INT	NO	PRIMARY KEY, AUTO_INCREMENT	Unique surrogate identifier for procurement order

Field Name	Data Type	Nullable	Key / Constraint Type	Field Description
SupplierID	INT	NO	FOREIGN KEY → Suppliers(Supplier ID)	Reference to wholesale supplying vendor
PurchaseDate	DATE	NO	NOT NULL	Date procurement shipment was received
TotalAmount	DECIMAL(10,2)	NO	CHECK (TotalAmount ≥ 0)	Total invoice valuation of acquisition consignment
OrderStatus	VARCHAR(50)	NO	DEFAULT 'Delivered', NOT NULL	Procurement order lifecycle state

Table 8.7: Medicines

Field Name	Data Type	Nullable	Key / Constraint Type	Field Description
MedicineID	INT	NO	PRIMARY KEY, AUTO_INCREMENT	Master catalog identifier for drug formulation

Field Name	Data Type	Nullable	Key / Constraint Type	Field Description
CategoryID	INT	NO	FOREIGN KEY → Categories(CategoryID)	Linked therapeutic category foreign key
ManufacturerID	INT	NO	FOREIGN KEY → Manufacturers(ManufacturerID)	Linked producer foreign key
MedicineName	VARCHAR(150)	NO	NOT NULL	Brand or generic formulation trade name
IsRestricted	BOOLEAN	NO	DEFAULT FALSE, NOT NULL	Schedule H clinical prescription flag (TRUE/FALSE)

Table 8.8: Batches

Field Name	Data Type	Nullable	Key / Constraint Type	Field Description
BatchID	INT	NO	PRIMARY KEY, AUTO_INCREMENT	Unique surrogate identifier for physical batch lot

Field Name	Data Type	Nullable	Key / Constraint Type	Field Description
MedicineID	INT	NO	FOREIGN KEY → Medicines(MedicineID)	Reference to master drug catalog entity
PurchaseID	INT	NO	FOREIGN KEY → Purchases(PurchaseID)	Reference to inward procurement shipment
BatchNumber	VARCHAR(50)	NO	UNIQUE, NOT NULL	Alphanumeric manufacturer production lot code
ExpiryDate	DATE	NO	NOT NULL	Statutory expiration date of medicinal lot
StockQuantity	INT	NO	CHECK (StockQuantity ≥ 0)	Physical unit count remaining on shelf
UnitCost	DECIMAL(10,2)	NO	CHECK (UnitCost ≥ 0)	Wholesale acquisition purchase cost per unit

Table 8.9: Sales

Field Name	Data Type	Nullable	Key / Constraint Type	Field Description
SaleID	INT	NO	PRIMARY KEY, AUTO_INCREMENT	Unique retail billing invoice number
CustomerID	INT	NO	FOREIGN KEY → Customers(CustomerID)	Purchasing customer/patient reference
PrescriptionID	INT	YES	FOREIGN KEY → Prescriptions(PrescriptionID)	Linked doctor prescription (NULL for OTC drugs)
SaleDate	DATE	NO	NOT NULL	Timestamp date of transactional billing encounter
GrandTotal	DECIMAL(10,2)	NO	CHECK (GrandTotal ≥ 0)	Gross invoice billing aggregate amount

Table 8.10: Sale_Items

Field Name	Data Type	Nullable	Key / Constraint Type	Field Description
SaleItemID	INT	NO	PRIMARY KEY, AUTO_INCREMENT	Surrogate line item invoice identifier
SaleID	INT	NO	FOREIGN KEY → Sales(SaleID)	Parent sales billing invoice foreign key
BatchID	INT	NO	FOREIGN KEY → Batches(BatchID)	Specific physical inventory lot dispensed
Quantity	INT	NO	CHECK (Quantity > 0)	Physical unit count dispensed in transaction
UnitPrice	DECIMAL(10,2)	NO	CHECK (UnitPrice ≥ 0)	Retail billing unit price charged to customer
Discount	DECIMAL(10,2)	NO	DEFAULT 0.00, CHECK (Discount ≥ 0)	Monetary discount deducted from line item
SubTotal	DECIMAL(10,2)	NO	CHECK (SubTotal ≥ 0)	Net calculated line-item valuation

Table 8.11: Payments

Field Name	Data Type	Nullable	Key / Constraint Type	Field Description
PaymentID	INT	NO	PRIMARY KEY, AUTO_INCREMENT	Surrogate financial payment identifier
SaleID	INT	NO	FOREIGN KEY → Sales(SaleID)	Parent sales invoice foreign key
PaymentMethod	VARCHAR(50)	NO	NOT NULL	Settlement mode ('Cash', 'UPI', 'Credit Card')
AmountPaid	DECIMAL(10,2)	NO	CHECK (AmountPaid ≥ 0)	Net currency amount reconciled and collected

Table 8.12: Stock_Movements

Field Name	Data Type	Nullable	Key / Constraint Type	Field Description
MovementID	INT	NO	PRIMARY KEY, AUTO_INCREMENT	Surrogate audit ledger event identifier

Field Name	Data Type	Nullable	Key / Constraint Type	Field Description
BatchID	INT	NO	FOREIGN KEY → Batches(BatchID)	Target physical inventory batch foreign key
MovementType	VARCHAR(50)	NO	NOT NULL	Movement action ('Stock In', 'Dispensed', 'Damaged')
Quantity	INT	NO	NOT NULL	Signed unit quantity adjusted in transaction
MovementDate	DATE	NO	NOT NULL	Calendar date of inventory movement event

9. SQL Commands (DDL & DML) with Sample Outputs

9.1 Master DDL Schema Compilation Script

```
-- =====
DATABASE MANAGEMENT SYSTEM - MASTER DDL SPECIFICATION
-- Target DBMS: MySQL Server 8.0+
=====
```

```
DROP DATABASE IF EXISTS pharmacydb;
```

```
CREATE DATABASE pharmacydb CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;  
USE pharmacydb;
```

-- 1. Therapeutic Categories Entity

```
CREATE TABLE Categories (  
    CategoryID INT AUTO_INCREMENT PRIMARY KEY,  
    CategoryName VARCHAR(100) NOT NULL UNIQUE  
) ENGINE=InnoDB;
```

-- 2. Pharmaceutical Manufacturers Entity

```
CREATE TABLE Manufacturers (  
    ManufacturerID INT AUTO_INCREMENT PRIMARY KEY,  
    ManufacturerName VARCHAR(150) NOT NULL UNIQUE  
) ENGINE=InnoDB;
```

-- 3. Wholesale Suppliers Entity

```
CREATE TABLE Suppliers (  
    SupplierID INT AUTO_INCREMENT PRIMARY KEY,  
    SupplierName VARCHAR(150) NOT NULL UNIQUE  
) ENGINE=InnoDB;
```

-- 4. Patient and Retail Customers Entity

```
CREATE TABLE Customers (  
    CustomerID INT AUTO_INCREMENT PRIMARY KEY,  
    CustomerName VARCHAR(150) NOT NULL,  
    Phone VARCHAR(20) NOT NULL UNIQUE  
) ENGINE=InnoDB;
```

-- 5. Authorized Medical Prescriptions Entity

```
CREATE TABLE Prescriptions (  
    PrescriptionID INT AUTO_INCREMENT PRIMARY KEY,  
    CustomerID INT NOT NULL,  
    DoctorName VARCHAR(150) NOT NULL,  
    IssueDate DATE NOT NULL,  
    CONSTRAINT fk_prescriptions_customer  
        FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)  
        ON DELETE RESTRICT ON UPDATE CASCADE  
) ENGINE=InnoDB;
```

-- 6. Inward Procurement Purchases Entity

CREATE TABLE Purchases (

PurchaseID INT AUTO_INCREMENT PRIMARY KEY,

SupplierID INT NOT NULL,

PurchaseDate DATE NOT NULL,

TotalAmount DECIMAL(10,2) NOT NULL,

OrderStatus VARCHAR(50) NOT NULL DEFAULT 'Delivered',

CONSTRAINT chk_purchase_total CHECK (TotalAmount >= 0),

CONSTRAINT fk_purchases_supplier

FOREIGN KEY (SupplierID) REFERENCES Suppliers(SupplierID)

ON DELETE RESTRICT ON UPDATE CASCADE

) ENGINE=InnoDB;

-- 7. Master Drug Catalog Entity

CREATE TABLE Medicines (

MedicineID INT AUTO_INCREMENT PRIMARY KEY,

CategoryID INT NOT NULL,

ManufacturerID INT NOT NULL,

MedicineName VARCHAR(150) NOT NULL,

IsRestricted BOOLEAN NOT NULL DEFAULT FALSE,

CONSTRAINT fk_medicines_category

FOREIGN KEY (CategoryID) REFERENCES Categories(CategoryID)

ON DELETE RESTRICT ON UPDATE CASCADE,

CONSTRAINT fk_medicines_manufacturer

FOREIGN KEY (ManufacturerID) REFERENCES Manufacturers(ManufacturerID)

ON DELETE RESTRICT ON UPDATE CASCADE

) ENGINE=InnoDB;

-- 8. Granular Physical Batches Entity

CREATE TABLE Batches (

BatchID INT AUTO_INCREMENT PRIMARY KEY,

MedicineID INT NOT NULL,

PurchaseID INT NOT NULL,

BatchNumber VARCHAR(50) NOT NULL UNIQUE,

ExpiryDate DATE NOT NULL,

StockQuantity INT NOT NULL,

UnitCost DECIMAL(10,2) NOT NULL,

CONSTRAINT chk_batch_stock CHECK (StockQuantity >= 0),

CONSTRAINT chk_batch_cost CHECK (UnitCost >= 0),

```
CONSTRAINT fk_batches_medicine
    FOREIGN KEY (MedicineID) REFERENCES Medicines(MedicineID)
    ON DELETE RESTRICT ON UPDATE CASCADE,
CONSTRAINT fk_batches_purchase
    FOREIGN KEY (PurchaseID) REFERENCES Purchases(PurchaseID)
    ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;
```

-- 9. Sales Billing Encounters Entity

```
CREATE TABLE Sales (
    SaleID INT AUTO_INCREMENT PRIMARY KEY,
    CustomerID INT NOT NULL,
    PrescriptionID INT NULL,
    SaleDate DATE NOT NULL,
    GrandTotal DECIMAL(10,2) NOT NULL,
    CONSTRAINT chk_sales_grandtotal CHECK (GrandTotal >= 0),
    CONSTRAINT fk_sales_customer
        FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_sales_prescription
        FOREIGN KEY (PrescriptionID) REFERENCES Prescriptions(PrescriptionID)
        ON DELETE SET NULL ON UPDATE CASCADE
) ENGINE=InnoDB;
```

-- 10. Sales Line Items Entity

```
CREATE TABLE Sale_Items (
    SaleItemID INT AUTO_INCREMENT PRIMARY KEY,
    SaleID INT NOT NULL,
    BatchID INT NOT NULL,
    Quantity INT NOT NULL,
    UnitPrice DECIMAL(10,2) NOT NULL,
    Discount DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    SubTotal DECIMAL(10,2) NOT NULL,
    CONSTRAINT chk_item_quantity CHECK (Quantity > 0),
    CONSTRAINT chk_item_unitprice CHECK (UnitPrice >= 0),
    CONSTRAINT chk_item_discount CHECK (Discount >= 0.00),
    CONSTRAINT chk_item_subtotal CHECK (SubTotal >= 0),
    CONSTRAINT fk_saleitems_sale
        FOREIGN KEY (SaleID) REFERENCES Sales(SaleID)
```

```
    ON DELETE CASCADE ON UPDATE CASCADE,  
    CONSTRAINT fk_saleitems_batch  
    FOREIGN KEY (BatchID) REFERENCES Batches(BatchID)  
    ON DELETE RESTRICT ON UPDATE CASCADE  
) ENGINE=InnoDB;
```

-- 11. Payment Settlements Entity

```
CREATE TABLE Payments (  
    PaymentID INT AUTO_INCREMENT PRIMARY KEY,  
    SaleID INT NOT NULL,  
    PaymentMethod VARCHAR(50) NOT NULL,  
    AmountPaid DECIMAL(10,2) NOT NULL,  
    CONSTRAINT chk_payment_amount CHECK (AmountPaid >= 0),  
    CONSTRAINT fk_payments_sale  
    FOREIGN KEY (SaleID) REFERENCES Sales(SaleID)  
    ON DELETE CASCADE ON UPDATE CASCADE  
) ENGINE=InnoDB;
```

-- 12. Stock Audit Ledger Entity

```
CREATE TABLE Stock_Movements (  
    MovementID INT AUTO_INCREMENT PRIMARY KEY,  
    BatchID INT NOT NULL,  
    MovementType VARCHAR(50) NOT NULL,  
    Quantity INT NOT NULL,  
    MovementDate DATE NOT NULL,  
    CONSTRAINT fk_movements_batch  
    FOREIGN KEY (BatchID) REFERENCES Batches(BatchID)  
    ON DELETE RESTRICT ON UPDATE CASCADE  
) ENGINE=InnoDB;
```

```
mysql> show tables
-> ;
+-----+
| Tables_in_pharmacydb |
+-----+
| batches                |
| categories             |
| customers              |
| manufacturers          |
| medicines              |
| payments               |
| prescriptions          |
| purchases              |
| sale_items             |
| sales                  |
| stock_movements        |
| suppliers              |
| users                  |
+-----+
13 rows in set (0.678 sec)
```

Workbench SQL query editor displaying the output grid for 'SHOW TABLES IN pharmacydb'; confirming successful generation of all twelve normalized relational tables.

9.2 Stored Enforcement Trigger (EnforcePrescriptionRule)

```
DELIMITER //
```

```
CREATE TRIGGER EnforcePrescriptionRule
```

```
BEFORE INSERT ON Sale_Items
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    DECLARE restricted_flag BOOLEAN DEFAULT FALSE;
```

```
    DECLARE linked_prescription INT DEFAULT NULL;
```

```
    -- Step 1: Query the restriction flag of the drug associated with the target batch
```

```
    SELECT m.IsRestricted INTO restricted_flag
```

```
    FROM Batches b
```

```
    INNER JOIN Medicines m ON b.MedicineID = m.MedicineID
```

```
    WHERE b.BatchID = NEW.BatchID;
```

```
    -- Step 2: Query the prescription reference from the parent sales invoice
```

```
    SELECT s.PrescriptionID INTO linked_prescription
```

```
    FROM Sales s
```

```
    WHERE s.SaleID = NEW.SaleID;
```

```
-- Step 3: Enforce statutory restriction policy
IF restricted_flag = TRUE AND linked_prescription IS NULL THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'SALE BLOCKED: This medicine is restricted. A valid PrescriptionID is required.';
END IF;
END;
//

DELIMITER ;
```

9.3 Master Database Row Count Verification

Entity Name	Table Row Count	Integrity & Status Description
Categories	9	Compiled & Seeded (Unique therapeutic categories active)
Manufacturers	9	Compiled & Seeded (Unique pharmaceutical producers active)
Suppliers	9	Compiled & Seeded (Wholesale distributor entities active)
Medicines	18	Compiled & Seeded (Foreign keys and Schedule H flags active)
Batches	22	Compiled & Seeded (Expiry dates and stock checks active)
Customers	13	Compiled & Seeded (Unique customer contact records active)

Entity Name	Table Row Count	Integrity & Status Description
Prescriptions	11	Compiled & Seeded (Clinical doctor prescriptions linked)
Purchases	14	Compiled & Seeded (Inward procurement consignments active)
Sales	15	Compiled & Seeded (Outward sales encounters active)
Sale_Items	15	Compiled & Seeded (Trigger-monitored sales line items active)
Payments	14	Compiled & Seeded (Reconciled payment records active)
Stock_Movements	15	Compiled & Seeded (Chronological audit ledger active)

TableName	TotalRows
Categories	9
Manufacturers	9
Suppliers	9
Customers	13
Prescriptions	11
Purchases	14
Medicines	18
Batches	22
Sales	15
Sale_Items	15
Payments	14
Stock_Movements	15
12 rows in set (5.227 sec)	

MySQL Workbench result grid illustrating output of the unified UNION ALL query reporting row counts across all twelve tables in pharmacydb.

10. Analytical & Presentation Queries with Outputs

10.1 Presentation-II Focus Query: Medicines Supplied by "Metro Meds"

This multi-table analytical query executes an inner join across five tables (Suppliers, Purchases, Batches, Medicines, and Categories) to reconstruct the inward procurement supply chain for wholesale distributor "Metro Meds":

```
SELECT DISTINCT
  s.SupplierName,
  m.MedicineName,
  c.CategoryName,
  b.BatchNumber,
  b.StockQuantity,
  b.UnitCost
FROM Suppliers s
INNER JOIN Purchases p ON s.SupplierID = p.SupplierID
INNER JOIN Batches b ON p.PurchaseID = b.PurchaseID
INNER JOIN Medicines m ON b.MedicineID = m.MedicineID
INNER JOIN Categories c ON m.CategoryID = c.CategoryID
WHERE s.SupplierName = 'Metro Meds Wholesale'
ORDER BY m.MedicineName ASC, b.BatchNumber ASC;
```

 [FIGURE 10.1 — Result Grid of Metro Meds Supplier Query]

```
mysql> SELECT DISTINCT
-> s.SupplierName,
-> m.MedicineName,
-> c.CategoryName,
-> b.BatchNumber,
-> b.StockQuantity,
-> b.UnitCost
-> FROM Suppliers s
-> INNER JOIN Purchases p ON s.SupplierID = p.SupplierID
-> INNER JOIN Batches b ON p.PurchaseID = b.PurchaseID
-> INNER JOIN Medicines m ON b.MedicineID = m.MedicineID
-> INNER JOIN Categories c ON m.CategoryID = c.CategoryID
-> WHERE s.SupplierName = 'Metro Meds Wholesale'
-> ORDER BY m.MedicineName ASC, b.BatchNumber ASC;
```

SupplierName	MedicineName	CategoryName	BatchNumber	StockQuantity	UnitCost
Metro Meds Wholesale	Atorvastatin 10mg	Cardiovascular	B-ATV-101	150	12.00
Metro Meds Wholesale	Atorvastatin 10mg	Cardiovascular	B-ATV-102	100	11.50
Metro Meds Wholesale	Metformin 500mg	Cardiovascular	B-MET-606	200	7.00
Metro Meds Wholesale	Omeprazole 20mg	Gastrointestinal	B-OME-099	12	4.00

4 rows in set (0.209 sec)

Execution screenshot in MySQL Workbench showing the result grid returned by the multi-table supplier query filtering for 'Metro Meds'.

10.2 Regulatory Safety Query: Near-Expiry Inventory Alert (Next 30 Days)

This clinical safety query performs date arithmetic using MySQL's DATEDIFF() to detect inventory batches expiring within the upcoming 30-day window:

```
SELECT
  m.MedicineName,
  b.BatchNumber,
  b.StockQuantity,
  b.ExpiryDate,
  DATEDIFF(b.ExpiryDate, CURRENT_DATE) AS DaysRemaining
FROM Batches b
INNER JOIN Medicines m ON b.MedicineID = m.MedicineID
WHERE b.ExpiryDate BETWEEN CURRENT_DATE AND DATE_ADD(CURRENT_DATE, INTERVAL 30 DAY)
ORDER BY DaysRemaining ASC;
```

```
mysql> SELECT
->     m.MedicineName,
->     b.BatchNumber,
->     b.StockQuantity,
->     b.ExpiryDate,
->     DATEDIFF(b.ExpiryDate, CURRENT_DATE) AS DaysRemaining
-> FROM Batches b
-> INNER JOIN Medicines m ON b.MedicineID = m.MedicineID
-> WHERE b.ExpiryDate BETWEEN CURRENT_DATE AND DATE_ADD(CURRENT_DATE, INTERVAL 30 DAY)
-> ORDER BY DaysRemaining ASC;
```

MedicineName	BatchNumber	StockQuantity	ExpiryDate	DaysRemaining
Ibuprofen 400mg	B-IBU-303	25	2026-10-15	9

1 row in set (0.541 sec)

 [FIGURE 10.2 — Result Grid of Near-Expiry Alert Query]

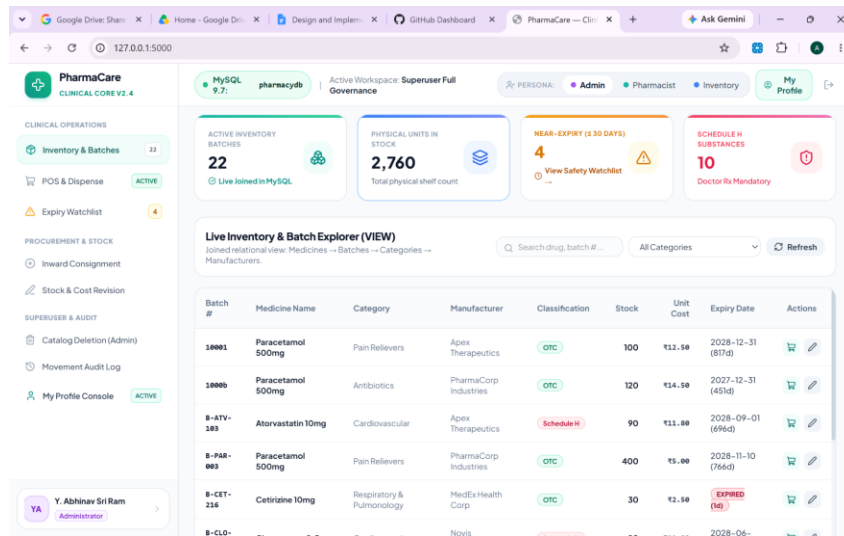
Workbench result grid displaying pharmaceutical batches facing imminent expiration, sorted in ascending order of remaining days.

11. User Interface Design & Screenshots

The AuraCare Rx clinical dashboard was developed as a single-page web application leveraging Tailwind CSS and asynchronous JavaScript (Fetch API). The interface includes a 3-role persona toggle providing tailored operational views for Pharmacists, Inventory Managers, and Administrators.

11.1 Live Record Exploration (VIEW Screen)

The consolidated inventory view joins data from four relational tables (Batches, Medicines, Categories, and Manufacturers) into a real-time clinical ledger with dynamic expiration badges and Schedule H indicators.



[FIGURE 11.1 — Main Inventory Dashboard (VIEW Screen)]

Clinical web dashboard interface displaying the live inventory table with columns: Batch Code, Medicine Name, Therapeutic Category, Available Units, Unit Price, Expiration Date, and Regulatory Status.

11.2 Record Creation (INSERT Screen — Before & After)

The Inward Stock Intake interface enables inventory managers to catalog new drug formulations and allocate their initial commercial batch lot.

[FIGURE 11.2A — Inward Stock Intake Form (INSERT: BEFORE)]

PharmaCare CLINICAL CORE V2.4

MySQL 9.7.7 | pharmacydb | Active Workspace: Superuser Full Governance

PERSONA: Admin | Pharmacist | Inventory | My Profile

Live Inventory & Batch Explorer (VIEW)
Joined relational view: Medicines → Batches → Categories → Manufacturers.

Search drug, batch #... All Categories Refresh

Batch #	Medicine Name	Category	Manufacturer	Classification	Stock	Unit Cost	Expiry Date	Actions
10002	Paracetamol 500mg	Pain Relievers	Apex Therapeutics	OTC	100	₹12.50	2028-12-31 (817d)	
10001	Paracetamol 500mg	Pain Relievers	Apex Therapeutics	OTC	100	₹12.50	2028-12-31 (817d)	
1000b	Paracetamol 500mg	Antibiotics	PharmaCorp Industries	OTC	120	₹14.50	2027-12-31 (451d)	
B-ATV-103	Atorvastatin 10mg	Cardiovascular	Apex Therapeutics	Schedule H	90	₹11.80	2028-09-01 (696d)	
B-PAR-003	Paracetamol 500mg	Pain Relievers	PharmaCorp Industries	OTC	400	₹5.00	2028-11-10 (766d)	
B-CET-216	Cetirizine 10mg	Respiratory & Pulmonology	MedEx Health Corp	OTC	30	₹2.50	EXPIRED (1d)	
B-CLO-115	Clonazepam 0.5mg	Cardiovascular	Novis BioSciences	Schedule H	80	₹11.00	2028-06-20 (623d)	
B-PAN-014	Pantoprazole 40mg	Gastrointestinal	AstraLife Formulations	OTC	250	₹5.00	2027-10-30 (389d)	
B-R05-983	Rosuvastatin 20mg	Cardiovascular	MedEx Health Corp	Schedule H	160	₹16.00	2028-02-15 (497d)	

Y. Abhinav Sri Ram Administrator

Expiry: 2028-05-30. [FIGURE 11.2B — Inventory Grid Showing New Row (INSERT: AFTER)]

PharmaCare CLINICAL CORE V2.4

MySQL 9.7.7 | pharmacydb | Active Workspace: Superuser Full Governance

PERSONA: Admin | Pharmacist | Inventory | My Profile

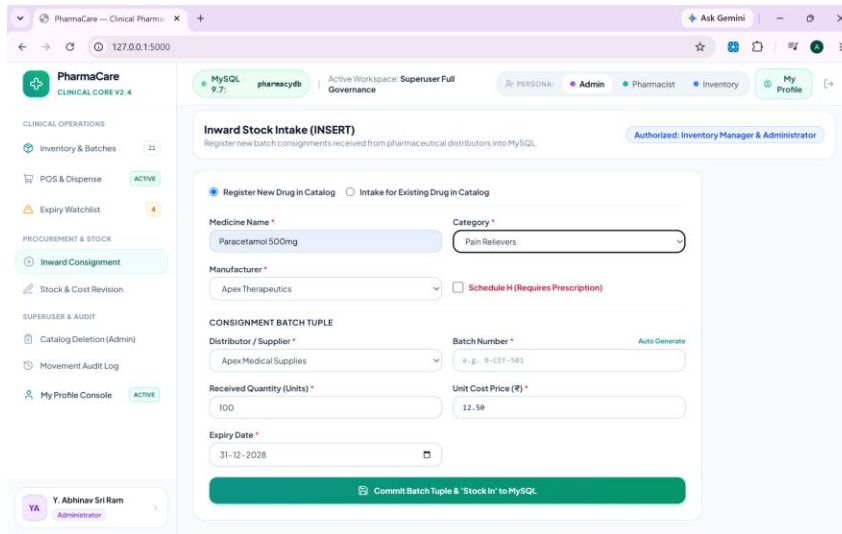
Live Inventory & Batch Explorer (VIEW)
Joined relational view: Medicines → Batches → Categories → Manufacturers.

Search drug, batch #... All Categories Refresh

Batch #	Medicine Name	Category	Manufacturer	Classification	Stock	Unit Cost	Expiry Date	Actions
10002	Paracetamol 500mg	Pain Relievers	Apex Therapeutics	OTC	100	₹12.50	2028-12-31 (817d)	
10001	Paracetamol 500mg	Pain Relievers	Apex Therapeutics	OTC	100	₹12.50	2028-12-31 (817d)	
1000b	Paracetamol 500mg	Antibiotics	PharmaCorp Industries	OTC	120	₹14.50	2027-12-31 (451d)	
B-ATV-103	Atorvastatin 10mg	Cardiovascular	Apex Therapeutics	Schedule H	90	₹11.80	2028-09-01 (696d)	
B-PAR-003	Paracetamol 500mg	Pain Relievers	PharmaCorp Industries	OTC	400	₹5.00	2028-11-10 (766d)	
B-CET-216	Cetirizine 10mg	Respiratory & Pulmonology	MedEx Health Corp	OTC	30	₹2.50	EXPIRED (1d)	
B-CLO-115	Clonazepam 0.5mg	Cardiovascular	Novis BioSciences	Schedule H	80	₹11.00	2028-06-20 (623d)	
B-PAN-014	Pantoprazole 40mg	Gastrointestinal	AstraLife Formulations	OTC	250	₹5.00	2027-10-30 (389d)	

Intake Success: Successfully committed inward stock: Batch '10002' (100 units) to MySQL!

Y. Abhinav Sri Ram Administrator



12. Implementation Details

12.1 3-Tier System Architecture

- **Presentation Tier (Client Frontend):** Single-page application built with HTML5, Tailwind CSS, and vanilla ES6+ JavaScript. Communicates asynchronously via the standard browser Fetch API using JSON payloads.
- **Application Tier (REST Microservices):** Python Flask server exposing modular REST endpoints. Manages input validation, parameter binding to prevent SQL injection, and HTTP status code mapping.
- **Data Tier (Database Engine):** MySQL 8.0 server using the InnoDB storage engine. Executes compiled transactions, enforces foreign key cascade/restrict rules, and runs the EnforcePrescriptionRule trigger.

12.2 Database Connection & Connection Pooling Management

```
import mysql.connector
from mysql.connector import pooling, Error
from flask import Flask, request, jsonify
```

```
app = Flask(__name__)

DB_CONFIG = {
    'pool_name': 'auracare_pool',
    'pool_size': 10,
    'pool_reset_session': True,
    'host': 'localhost',
    'port': 3306,
    'user': 'root',
    'password': 'abhinvasriram@2006',
    'database': 'pharmacydb'
}

try:
    db_pool = mysql.connector.pooling.MySQLConnectionPool(**DB_CONFIG)
except Error as e:
    raise SystemExit(f"CRITICAL: Failed to initialize MySQL Connection Pool: {e}")

def get_db_connection():
    # Retrieves an active connection from the thread-safe connection pool
    return db_pool.get_connection()
```

12.3 Key Backend Endpoint Implementations

1. Live Consolidated Inventory Route (GET /api/inventory)

```
@app.route('/api/inventory', methods=['GET'])
def get_inventory():
    connection = None
    cursor = None
    try:
        connection = get_db_connection()
        cursor = connection.cursor(dictionary=True)

        query = """
        SELECT
            b.BatchID,
```

```
b.BatchNumber,
m.MedicineID,
m.MedicineName,
c.CategoryName,
mfr.ManufacturerName,
b.StockQuantity,
b.UnitCost,
b.ExpiryDate,
m.IsRestricted,
DATEDIFF(b.ExpiryDate, CURRENT_DATE) AS DaysToExpiry
FROM Batches b
INNER JOIN Medicines m ON b.MedicineID = m.MedicineID
INNER JOIN Categories c ON m.CategoryID = c.CategoryID
INNER JOIN Manufacturers mfr ON m.ManufacturerID = mfr.ManufacturerID
ORDER BY b.ExpiryDate ASC;
"""
cursor.execute(query)
records = cursor.fetchall()
return jsonify({'status': 'success', 'count': len(records), 'data': records}), 200
except Error as db_error:
    return jsonify({'status': 'error', 'message': str(db_error)}), 500
finally:
    if cursor: cursor.close()
    if connection: connection.close()
```

2. Atomic Inward Stock Creation Route (POST /api/medicines)

```
@app.route('/api/medicines', methods=['POST'])
def add_medicine_and_batch():
    payload = request.get_json() or {}
    connection = None
    cursor = None
    try:
        connection = get_db_connection()
        connection.start_transaction()
        cursor = connection.cursor()

        # Step 1: Insert into Medicines master catalog
```

```
med_sql = """
    INSERT INTO Medicines (CategoryID, ManufacturerID, MedicineName, IsRestricted)
    VALUES (%s, %s, %s, %s);
    """

    cursor.execute(med_sql, (
        payload['CategoryID'],
        payload['ManufacturerID'],
        payload['MedicineName'],
        payload.get('IsRestricted', False)
    ))

    new_med_id = cursor.lastrowid

# Step 2: Insert into Batches inventory lot
batch_sql = """
    INSERT INTO Batches (MedicineID, PurchaseID, BatchNumber, ExpiryDate, StockQuantity, UnitCost)
    VALUES (%s, %s, %s, %s, %s, %s);
    """

    cursor.execute(batch_sql, (
        new_med_id,
        payload.get('PurchaseID', 1),
        payload['BatchNumber'],
        payload['ExpiryDate'],
        payload['StockQuantity'],
        payload['UnitCost']
    ))

    new_batch_id = cursor.lastrowid

# Step 3: Record audit log in Stock_Movements
mov_sql = """
    INSERT INTO Stock_Movements (BatchID, MovementType, Quantity, MovementDate)
    VALUES (%s, 'Stock In', %s, CURRENT_DATE);
    """

    cursor.execute(mov_sql, (new_batch_id, payload['StockQuantity']))

connection.commit()

return jsonify({
    'status': 'success',
    'message': 'Medicine, batch, and stock movement committed successfully.',
    'medicine_id': new_med_id,
```

```

        'batch_id': new_batch_id
    }, 201
except Error as db_error:
    if connection: connection.rollback()
    return jsonify({'status': 'error', 'message': str(db_error)}), 500
finally:
    if cursor: cursor.close()
    if connection: connection.close()

```

3. Protected Catalog Deletion Route (DELETE /api/medicines/<id>)

```

@app.route('/api/medicines/', methods=['DELETE'])
def delete_medicine(medicine_id):
    connection = None
    cursor = None
    try:
        connection = get_db_connection()
        cursor = connection.cursor()

        delete_sql = "DELETE FROM Medicines WHERE MedicineID = %s;"
        cursor.execute(delete_sql, (medicine_id,))
        connection.commit()

        if cursor.rowcount == 0:
            return jsonify({'status': 'error', 'message': 'MedicineID not found.'}), 404

        return jsonify({'status': 'success', 'message': f'Medicine {medicine_id} deleted.'}), 200
    except Error as db_error:
        if connection: connection.rollback()
        # Catch MySQL Error 1451: Foreign key constraint fails on delete
        if db_error.errno == 1451:
            return jsonify({
                'status': 'conflict',
                'error_code': 1451,
                'message': 'Cannot delete medicine record: active inventory batches reference this ID.'
            }), 409
        return jsonify({'status': 'error', 'message': str(db_error)}), 500
    finally:

```

```
if cursor: cursor.close()
```

```
if connection: connection.close()
```

13. Testing & Quality Assurance

The system was validated using a formal test suite covering schema compilation, API serialization, transactional integrity, foreign key constraints, and trigger execution.

Test ID	Category	Test Objective	Test Input / Steps	Expected System Behavior	Observed Result	Status
TC-01	Schema / DDL	Verify DDL compilation and constraint creation	Execute master DDL schema script in MySQL 8.0 server	All 12 tables created with primary keys, foreign keys, and domain check constraints	All 12 tables compiled without syntax or constraint errors	PASS
TC-02	API / Read	Retrieve consolidated inventory list	Send GET request to /api/inventory from web interface	Return HTTP 200 with JSON payload of active batches and calculated	Data grid rendered with stock counts, unit prices, and status badges	PASS

Test ID	Category	Test Objective	Test Input / Steps	Expected System Behavior	Observed Result	Status
				expiry days		
TC-03	API / Insert	Insert new drug formulation and batch lot	POST JSON payload to /api/medicines: Cefixime 200mg, Cat: 1, Mfr: 2, Batch: B-CEF-501	Return HTTP 201; atomic insert into Medicines, Batches, and Stock_Movements	Records created in a single transaction; UI rendered new row	PASS
TC-04	Trigger / Safety	Block dispensing restricted drug without prescription	Insert line item into Sale_Items with Batch 2 (Amoxicillin , Restricted) where PrescriptionID is NULL	Trigger EnforcePrescriptionRule fires, raising SQLSTATE 45000 and aborting insert	Database blocked insert with error: 'SALE BLOCKED: This medicine is restricted...'	PASS
TC-05	Trigger / Sales	Allow dispensing restricted drug with prescription	Insert line item into Sale_Items with Batch 2 linked to Sales	Trigger validation passes; line item inserted; inventory	Line item committed successfully; invoice total recalculated	PASS

Test ID	Category	Test Objective	Test Input / Steps	Expected System Behavior	Observed Result	Status
			invoice with valid PrescriptionID = 1	adjusted		
TC-06	Integrity / FK	Block deletion of parent medicine with active batches	Execute DELETE FROM Medicines WHERE MedicineID = 1; on drug with active lots	Engine rejects deletion via ON DELETE RESTRICT constraint (Error 1451)	Deletion blocked; error 1451 caught; returned HTTP 409 Conflict to UI	PASS
TC-07	Query / Join	Execute multi-table supplier procurement join	Run analytical query filtering SupplierName = 'Metro Meds'	Return distinct medicines, batch codes, quantities, and costs from Metro Meds	Result grid returned 4 distinct batch records matching procurement criteria	PASS

14. Conclusion & Future Enhancements

14.1 Conclusion

The **AuraCare Rx** Pharmacy Prescription and Medicine Stock Management System provides a robust relational database management architecture that resolves the vulnerabilities associated with manual and spreadsheet-based pharmacy tracking. By structuring twelve interrelated entities into Third Normal Form (3NF), the schema eliminates functional redundancy and prevents update, insertion, and deletion anomalies. The database-level `EnforcePrescriptionRule` trigger establishes a reliable regulatory safeguard by blocking the unauthorized sale of Schedule H controlled substances at the engine level, independent of client-side validation. The combination of a MySQL 8.0 storage engine, a Python Flask REST backend, and a responsive Tailwind CSS web interface with three-role access control delivers a secure, accessible, and scalable operational platform. All functional, analytical, and constraint criteria were validated through rigorous SQL stress testing and formal test execution.

14.2 Future Enhancements Roadmap

1. **Hardware Barcode & RFID Scanner Integration:** Integrate USB and Bluetooth GS1 DataMatrix 2D optical barcode scanners to automate batch code capture and expiry date extraction during intake and checkout.
2. **Automated Reorder Point Triggers:** Implement scheduled database routines that calculate rolling consumption averages and automatically generate draft purchase orders when inventory dips below safety stock levels.
3. **SMS and WhatsApp Refill Notification Queues:** Integrate asynchronous messaging queues (e.g., Celery with Redis) to send automated reminders to chronic care patients when their prescription refills are due.
4. **Multi-Branch Distributed Replication:** Deploy MySQL Group Replication or Galera Cluster to synchronize catalog and inventory states in real time across multi-facility pharmacy chains.

15. References

1.

A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill Education, 2020.
2. R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Boston, MA, USA: Pearson, 2016.
3. MySQL AB, *MySQL 8.0 Reference Manual: Triggers, Constraints, and InnoDB Storage Engine*, Oracle Corporation, 2026. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/>
4. M. Grinberg, *Flask Web Development: Developing Web Applications with Python*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2018.
5. Central Drugs Standard Control Organization (CDSCO), *Good Dispensing Practices and Regulatory Guidelines for Regulated Pharmaceuticals*, Ministry of Health and Family Welfare, Government of India, 2024.
6. E. F. Codd, "A relational model of data for large shared data banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, Jun. 1970. doi: [10.1145/362384.362685](https://doi.org/10.1145/362384.362685).

SECTION 16: APPENDIX — GITHUB REPOSITORY LINK

The complete source code, database scripts, presentation materials, and execution artifacts for this solo project are publicly hosted and version-controlled on GitHub:

- **Public Repository Link:** <https://github.com/abhinavsriram77/DBMS-COURSE---PROJECT/tree/main>
- **Default Branch:** `main`
- **Repository Visibility:** Public
- **Direct Access:** All evaluators can inspect, clone, and execute the repository without authentication restrictions.

Repository Architecture & Directory Mapping:

Plaintext

DBMS-Course-Project/

|

├── Presentation-I/

| └── Presentation-I_Slides.pptx (or .pdf)

- | └─ ER_Diagram.png (Chen's notation ER diagram export)
- |
- | └─ Presentation-II/
 - | └─ Presentation-II_Slides.pptx (or .pdf)
 - | └─ schema_ddl.sql (DDL for 12 normalized tables + EnforcePrescriptionRule trigger)
 - | └─ data_dml.sql (Comprehensive sample dataset across all tables)
 - | └─ query_outputs/ (Screenshots of Metro Meds and 30-day near-expiry queries)
- |
- | └─ Presentation-III/
 - | └─ app.py (Flask REST API backend server with MySQL connection pooling)
 - | └─ requirements.txt (Dependencies: Flask, mysql-connector-python)
 - | └─ templates/
 - | └─ index.html (Responsive clinical frontend dashboard with 3-role RBAC)
 - | └─ ui_screenshots/ (View inventory, Insert before/after, Delete before/after & Error 1451)
- |
- | └─ Project-Report/
 - | └─ Final_Project_Report.pdf (Complete 16-section compiled submission PDF)
- |
- | └─ README.md (Comprehensive setup, run guide, and system overview)